5. Prototype Design Pattern in Java

5.1 Introduction

The **Prototype Design Pattern** is a **creational pattern** from the Gang of Four (GoF) that allows you to create new objects by copying existing ones, rather than instantiating new ones from scratch.

This is especially useful when:

- Object creation is expensive (e.g., network, I/O)
- You want to preserve complex configuration

5.2 Problem it Solves

When creating new objects is costly or complex, creating a new object by copying an existing one is more efficient.

5.3 What is Prototype Pattern?

- Uses a prototype interface that includes a clone() method
- Concrete classes implement the interface and clone themselves

Java has a built-in Cloneable interface and clone() method in Object.

5.4 Shallow vs Deep Copy

Shallow Copy

- Copies object structure, but **not referenced objects**
- Both original and copy share the same nested objects

Deep Copy

- Copies **entire object graph**, including all referenced objects
- Original and copy are fully independent

Type	Shared References?	Performance	Usage
Shallow Copy	Yes	Faster	When nested objects don't change independently
Deep Copy	No	Slower	When complete independence is required

5.5 Basic Implementation Structure

Example1:

```
package com.mannemlokesh;

class PrototypeObject implements Cloneable {
	private String value;
```

```
	public PrototypeObject(String value) {
		this.value = value;
	}

	@Override
	protected Object clone() throws CloneNotSupportedException {
		return super.clone(); // Shallow copy
	}

	public void show() {
		System.out.println("Value: " + value);
	}
}

public class Example1BasicCloneDemo {
	public static void main(String[] args) throws CloneNotSupportedException {
		PrototypeObject obj1 = new PrototypeObject("Hello");
		PrototypeObject obj2 = (PrototypeObject) obj1.clone();

		obj1.show();
		obj2.show();
	}
}
```

Output:

```
Value: Hello
Value: Hello
```

5.6 Step-by-Step Examples

Example2: Beginner: Shape Cloning (Shallow Copy)

```
package com.mannemlokesh;

interface Shape extends Cloneable {
	Shape clone();

	void draw();
}

class Circle implements Shape {
	private int radius;

	public Circle(int radius) {
		this.radius = radius;
	}

	public void draw() {
		System.out.println("Drawing Circle with radius: " + radius);
	}

	public Shape clone() {
		try {
			return (Shape) super.clone();
		} catch (CloneNotSupportedException e) {
			throw new AssertionError();
		}
```

```
	}
}

public class Example2ShapeApp {
	public static void main(String[] args) {
		Shape circle = new Circle(5);
		Shape copy = circle.clone();

		circle.draw();
		copy.draw();
	}
}
```

Output:

```
Drawing Circle with radius: 5
Drawing Circle with radius: 5
```

Example3: Intermediate: Document Template (Deep Copy)

```
package com.mannemlokesh;

class Document implements Cloneable {
    private String header;
    public StringBuilder body;

    public Document(String header, StringBuilder body) {
        this.header = header;
        this.body = body;
    }

    public void print() {
        System.out.println("\n--- Document ---");
        System.out.println("Header: " + header);
        System.out.println("Body: " + body);
    }

    @Override
    protected Document clone() {
        try {
            Document copy = (Document) super.clone();
            copy.body = new StringBuilder(this.body.toString()); // Deep copy
            return copy;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }
}

public class Example3DocumentApp {
    public static void main(String[] args) {
        Document doc1 = new Document("Invoice", new StringBuilder("Items list"));
        Document doc2 = doc1.clone();

        doc1.print();
        doc2.print();

        doc2.body.append(" - Updated");

        System.out.println("\nAfter modification:");
```

```
        doc1.print();
        doc2.print();
    }
}
```

Output:

```
--- Document ---
Header: Invoice
Body: Items list

--- Document ---
Header: Invoice
Body: Items list

After modification:

--- Document ---
Header: Invoice
Body: Items list

--- Document ---
Header: Invoice
Body: Items list - Updated
```

Example4: Advanced: Game Character Prototype (Deep Copy)

```
package com.mannemlokesh;

class GameCharacter implements Cloneable {
	private String name;
	private int health;
	private String[] inventory;

	public GameCharacter(String name, int health, String[] inventory) {
		this.name = name;
		this.health = health;
		this.inventory = inventory.clone(); // Deep copy
	}

	public void display() {
		System.out.println("Character: " + name + ", Health: " + health + ",
                                    Inventory: " + String.join(", ", inventory));
	}

	@Override
	public GameCharacter clone() {
		try {
			GameCharacter copy = (GameCharacter) super.clone();
			copy.inventory = this.inventory.clone(); // Deep copy of array
			return copy;
		} catch (CloneNotSupportedException e) {
			throw new AssertionError();
		}
	}
}
```

```
public class Example4GameApp {
	public static void main(String[] args) {
		GameCharacter original = new GameCharacter("Warrior", 100, new
                                                    String[] { "Sword", "Shield" });
		GameCharacter clone = original.clone();

		original.display();
		clone.display();
	}
}
```

Output:

```
Character: Warrior, Health: 100, Inventory: Sword, Shield
Character: Warrior, Health: 100, Inventory: Sword, Shield
```

5.7 Pros and Cons

✅ **Pros:**

- Avoids cost of object creation
- Simplifies object construction
- Preserves object configuration
- Runtime cloning flexibility

❌ **Cons:**

- Cloneable interface is flawed (no clone() method in it)
- super.clone() only performs shallow copy by default
- Deep copying must be handled manually

5.8 When to Use / Not to Use

Use When:

- Object creation is costly
- You need many similar objects
- You want to isolate object construction logic

Avoid When:

- Object is simple and inexpensive to create
- Deep copies are hard to manage

5.9 Differences Between Shallow and Deep Copy

Feature	Shallow Copy	Deep Copy
References	Shared	Independent
Performance	Fast	Slower
Complexity	Simple	More complex

Use Case	Nested objects are immutable or not edited	When complete separation is required
Example	super.clone()	Manual cloning of nested objects/arrays

5.10 Summary

Feature	Description
Cloneable Interface	Marker interface used for cloning
super.clone()	Provides shallow copy from Object class
Deep Copy	Must be handled manually for references
Use Cases	Prototypes, templates, cached instances

The Prototype Pattern is ideal when:

- You have complex object setups
- You need to generate many instances with minimal cost
- You need runtime object duplication